COMPLETE IMPLEMENTATION BLUEPRINT:

# Fundraising Platform — Azure & DevOps

Complete cloud implementation notes • identity • Azure SQL • containers • messaging • CI/CD • Kubernetes • security • observability

## INDEX — Main Topics & Important Subtopics

Study flow: Azure organization → identity → secrets/data → containers → messaging/gateways → CI/CD/IaC → Kubernetes → observability/security → production operations.


- Purpose & Final Azure Architecture
- Azure Organization & Environments
- Microsoft Entra ID, App Registrations & Managed Identity
- Key Vault & Configuration
- Azure SQL & Passwordless Access
- Docker & Local Containers
- Azure Container Registry
- Hosting: App Service vs Container Apps vs AKS
- Azure Container Apps, Revisions & Jobs
- Azure Functions
- Service Bus, DLQ & Outbox
- API Management
- Front Door, WAF & Load Balancing
- VNet, Private Endpoints & DNS
- Git, GitHub & Pull Requests
- GitHub Actions CI/CD & OIDC
- Bicep / Infrastructure as Code
- Deploying Fundraising, Payment & Reporting
- Angular Deployment
- Kubernetes / AKS
- Environment Promotion & Database Migrations
- Blue/Green, Canary & Rollback
- Observability, Azure Monitor & Health
- Security Architecture & RBAC
- High Availability & Disaster Recovery
- Cost Management
- Azure Build Stages
- Production-Like Architecture
- Complete Azure Topic Checklist
- Final Build Sequence & Memory Map


## Purpose of This Document

This document takes the system we already designed:

EXAMPLE / FLOW

```
Fundraising Service
        |
        v
FundraisingDb

Payment Service
        |
        v
PaymentDb

Reporting Service
        |
        v
ReportingDb
```

and explains how to deploy, secure, scale, monitor, and operate it in Microsoft Azure.

The goal is not just:

EXAMPLE / FLOW

```
"Deploy a .NET API to Azure."
```

The goal is to understand the complete cloud architecture:

EXAMPLE / FLOW

```
Microsoft Entra tenant
        |
        v
Azure subscriptions
        |
        v
Resource groups
        |
        v
Networking
        |
        v
Identity / App registrations
        |
        v
Azure SQL
        |
        v
Key Vault
        |
        v
Azure Container Registry
        |
        v
Docker
```

EXAMPLE / FLOW — continued

```
        |
        v
Container Apps / AKS
        |
        v
Service Bus
        |
        v
Azure Functions / Workers
        |
        v
API Management
        |
        v
Front Door
        |
        v
Angular
        |
        v
Monitoring / Application Insights
        |
        v
GitHub Actions CI/CD
```

This document also explains:

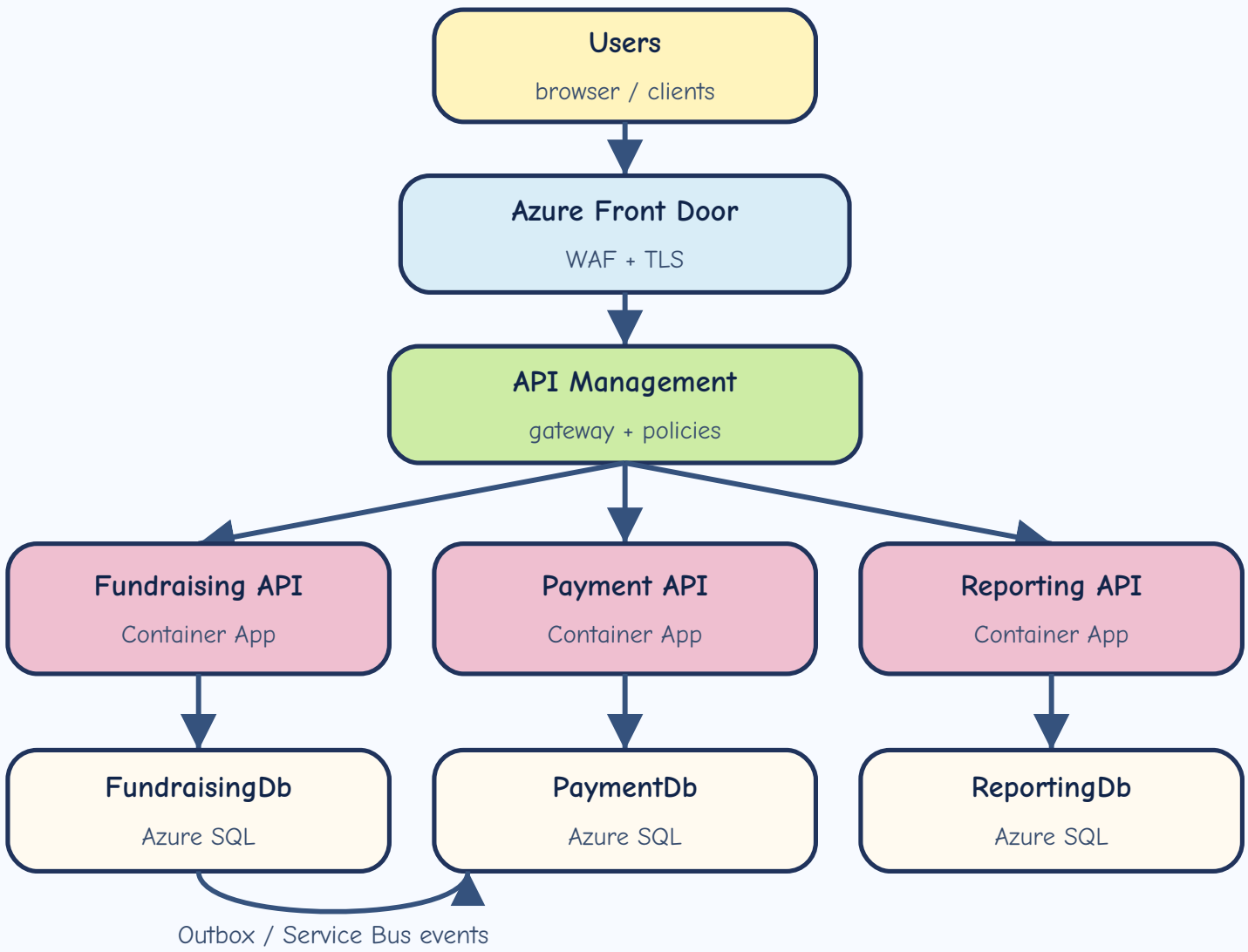
- what each Azure service does,
- why this project needs it,
- where Fundraising, Payments, and Reporting use it,
- what belongs in development versus production,
- how environments should be separated,
- how identity works,
- how secrets should be stored,
- how Docker images are built,
- how GitHub Actions deploys them,
- how Kubernetes fits,
- how APIs communicate,
- how the system scales,
- how failed deployments are rolled back,
- and how to monitor the entire platform.

## The Final Azure Architecture

The target architecture is:

DIAGRAM

## Fundraising Platform — Azure Request & Data Flow




Front Door and API Management protect the edge; each service owns its Azure SQL database and communicates asynchronously where appropriate.

EXAMPLE / FLOW

```
                          USERS
                            |
                            v
                     Azure Front Door
                       + WAF / TLS
                            |
                            v
                   Azure API Management
                            |
          +-------------------+-------------------+
          |                   |                   |
          v                   v                   v
Fundraising API         Payment API         Reporting API
Container App           Container App       Container App
          |                   |                   |
          |                   |                   |
          v                   v                   v
FundraisingDb           PaymentDb           ReportingDb
Azure SQL               Azure SQL           Azure SQL
          |
          v
     Outbox Worker
          |
          v
  Azure Service Bus
```

EXAMPLE / FLOW — continued

```
          |
      +---+---------+
      |             |
      v             v
Payment Worker   Reporting/ETL
      |
      v
External Payment Provider


Angular SPA
    |
    v
Static Web Apps /
Storage + Front Door
or suitable web hosting


All services
    |
    +---- Managed Identity
    |
    +---- Key Vault
    |
    +---- Application Insights
```

EXAMPLE / FLOW — continued

```
    |
    +---- Log Analytics
    |
    +---- Azure Monitor
```

CI/CD:

EXAMPLE / FLOW

```
Developer
    |
    v
GitHub
    |
    v
Pull Request
    |
    +---- Build
    +---- Unit Tests
    +---- Integration Tests
    +---- Security Checks
    |
    v
Merge to main
    |
    v
GitHub Actions
    |
    v
OIDC login to Azure
    |
    v
Build Docker image
    |
```

EXAMPLE / FLOW — continued

```
    v
Azure Container Registry
    |
    v
Deploy new Container App revision
```

# UNDERSTANDING AZURE ORGANIZATION

# Microsoft Entra Tenant

## What Is a Tenant?

A Microsoft Entra tenant is the identity boundary for an organization.

It contains identities such as:

EXAMPLE / FLOW

```
Users
Groups
Enterprise applications
App registrations
Managed identities
Service principals
```

Think:

EXAMPLE / FLOW

```
Microsoft Entra Tenant
=
Identity organization
```

Example:

EXAMPLE / FLOW

```
contoso.onmicrosoft.com
```

may contain:

EXAMPLE / FLOW

```
Sarah Johnson
David Wilson
Angular SPA application
Fundraising API application
Payment API application
Managed identities
```

Your Azure subscription is associated with a Microsoft Entra tenant.

## Tenant Is Not the Same as Your Application Tenant

This project also contains:

EXAMPLE / FLOW

```
core.Tenant
```

in `FundraisingDb`.

Do not confuse these.

EXAMPLE / FLOW

```
Microsoft Entra Tenant
→ Azure identity organization

FundraisingDb core.Tenant
→ customer/nonprofit using our SaaS application
```

Example:

EXAMPLE / FLOW

```
Microsoft Entra Tenant
OurCompany.onmicrosoft.com
```

may run the SaaS platform for:

EXAMPLE / FLOW

```
Application Tenant 1
ABC Children's Foundation

Application Tenant 2
XYZ University
```

Those are completely different concepts.

## Azure Subscription

A subscription is primarily:

EXAMPLE / FLOW

```
billing
+
resource-management
+
quota
+
RBAC boundary
```

Conceptually:

EXAMPLE / FLOW

```
Entra Tenant
     |
     +---- Development Subscription
     |
     +---- Production Subscription
```

For a personal learning project, one subscription is sufficient.

For a more production-like organization, separate:

EXAMPLE / FLOW

```
NonProduction Subscription

Production Subscription
```

This gives stronger separation.

## Resource Groups

A resource group organizes Azure resources that belong to a lifecycle/environment.

Recommended learning layout:

EXAMPLE / FLOW

```
rg-fundraising-dev
rg-fundraising-test
rg-fundraising-prod
```

Production-style organizations may instead use multiple resource groups by responsibility:

EXAMPLE / FLOW

```
rg-platform-prod-network

rg-platform-prod-data

rg-platform-prod-apps

rg-platform-prod-observability
```

Do not create one resource group per tiny resource without a reason.

## Environment Strategy

You should have at least:

EXAMPLE / FLOW

```
Development
Test / QA
Production
```

A stronger system may have:

EXAMPLE / FLOW

```
Local
Development
Integration
QA
Staging
Production
```

For this project:

EXAMPLE / FLOW

```
Local
Dev
Staging
Prod
```

is enough.

## Why Separate Environments?

Never test a breaking database migration directly against production.

Each environment should have its own:

EXAMPLE / FLOW

```
Container Apps
Databases
Service Bus
Key Vault
Configuration
Monitoring
```

Example:

EXAMPLE / FLOW

```
fundraising-api-dev
fundraising-api-staging
fundraising-api-prod
```

and:

EXAMPLE / FLOW

```
fundraisingdb-dev
fundraisingdb-staging
fundraisingdb-prod
```

## Environment Naming Convention

Choose one convention early.

Example:

EXAMPLE / FLOW

```
<application>-<component>-<environment>
```

Examples:

EXAMPLE / FLOW

```
fp-fundraising-api-dev

fp-payments-api-dev

fp-reporting-api-dev

fp-servicebus-dev

fp-kv-dev

fp-acr-dev
```

Production:

EXAMPLE / FLOW

```
fp-fundraising-api-prod
```

Consistency becomes important once you have many Azure resources.

## MICROSOFT ENTRA ID

## Why We Need Entra ID

Microsoft Entra ID will handle:

EXAMPLE / FLOW

```
Angular user login

API access tokens

roles

scopes

service identities

managed identities

Azure resource authorization
```

Do not implement your own:

EXAMPLE / FLOW

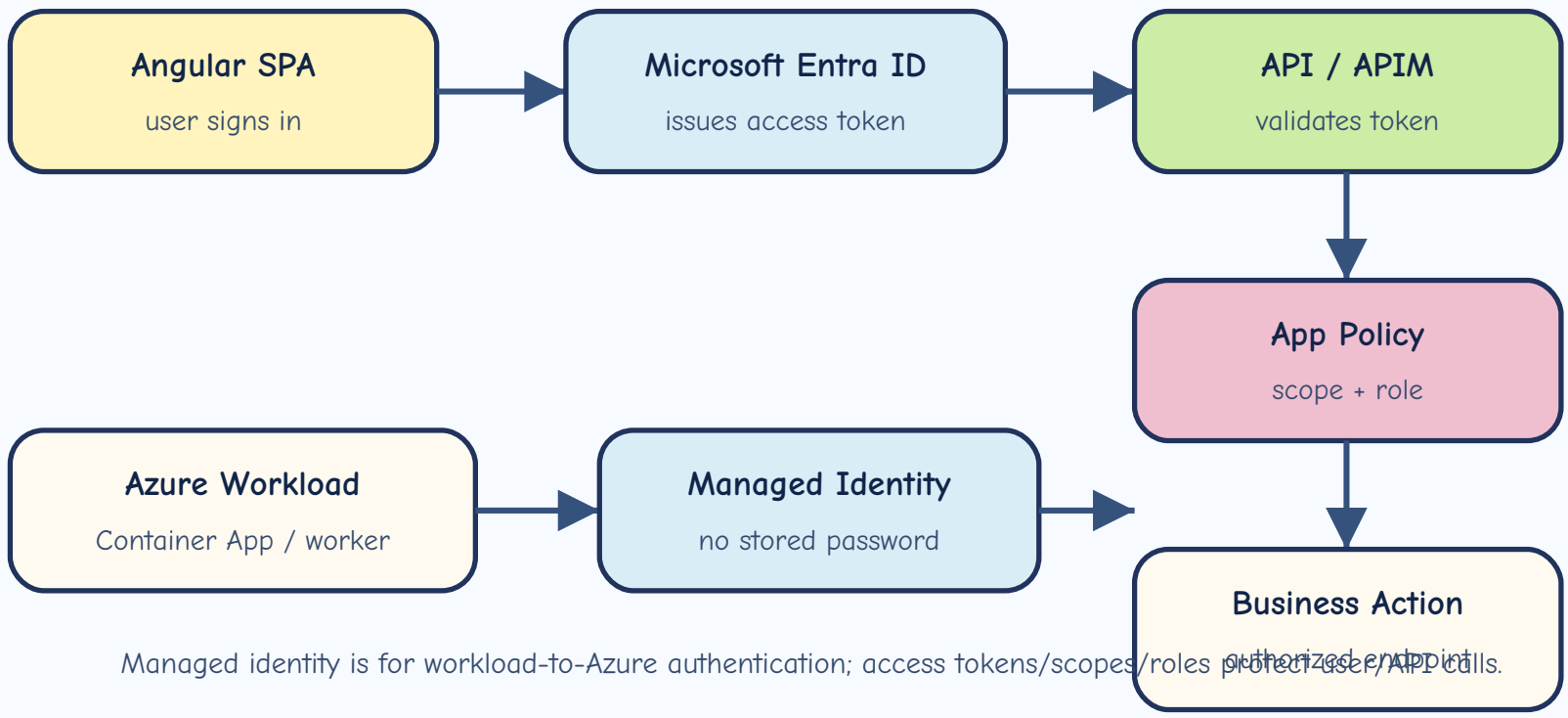
```
username/password database
custom JWT issuer
home-grown token system
```

for this project.

## Main Identity Architecture

DIAGRAM

Identity: Users vs Azure Workloads




Managed identity is for workload-to-Azure authentication; access tokens/scopes/roles protect user/API calls.

Human/API delegated access and managed identity solve different authentication paths but share Microsoft Entra as the identity authority.

EXAMPLE / FLOW

```
Angular
   |
   v
Microsoft Entra ID
   |
   | access token
   v
API Management
   |
   v
ASP.NET Core API
```

The API validates:

EXAMPLE / FLOW

```
issuer
audience
signature
expiration
scope / role
```

## App Registrations We Need

Recommended:

EXAMPLE / FLOW

```
1. Angular SPA

2. Fundraising API

3. Payment API

4. Reporting API
```

Optionally later:

EXAMPLE / FLOW

```
5. Admin portal

6. External partner application
```

## Why Separate API App Registrations?

Because each microservice should own its API permissions.

Example:

EXAMPLE / FLOW

```
Fundraising API
scope:
fundraising.access
```

Payment:

EXAMPLE / FLOW

```
Payment API
scope:
payments.access
```

Reporting:

EXAMPLE / FLOW

```
Reporting API
scope:
reporting.read
```

This keeps authorization boundaries clear.

## Register Fundraising API

In:

EXAMPLE / FLOW

```
Microsoft Entra admin center
    |
    v
App registrations
    |
    v
New registration
```

create:

EXAMPLE / FLOW

```
Name:
Fundraising API
```

For an internal enterprise learning project choose:

EXAMPLE / FLOW

```
Accounts in this organizational directory only
```

unless you intentionally want a multi-organization identity model.

For a Web API app registration, a redirect URI is not normally required just to protect the API. Microsoft's current API registration guidance follows this model and then exposes API scopes from the registration.

## Expose an API

Fundraising API needs a resource identifier.

Default:

EXAMPLE / FLOW

```
api://<fundraising-api-client-id>
```

Then create a delegated scope such as:

EXAMPLE / FLOW

```
fundraising.access
```

Full scope becomes something like:

EXAMPLE / FLOW

```
api://<client-id>/fundraising.access
```

Microsoft Entra protected APIs expose delegated permissions through scopes; clients request those scopes and receive them in access tokens when consent is available.

## Payment API Registration

Create another registration:

EXAMPLE / FLOW

```
Payment API
```

Expose:

EXAMPLE / FLOW

```
payments.access
```

Potential administrative scope:

EXAMPLE / FLOW

```
payments.refund
```

However, for employee permission models, app roles/policies are often better than creating an excessive number of OAuth scopes.

## Reporting API Registration

Expose:

EXAMPLE / FLOW

```
reporting.read
```

Reporting should generally be read-oriented.

## Angular SPA Registration

Create:

EXAMPLE / FLOW

```
Fundraising Angular SPA
```

Platform:

EXAMPLE / FLOW

```
Single-page application
```

Development redirect:

EXAMPLE / FLOW

```
http://localhost:4200
```

Production later:

EXAMPLE / FLOW

```
https://app.example.com
```

Angular will use:

EXAMPLE / FLOW

```
MSAL
```

to authenticate users and acquire access tokens.

## Angular Permission Configuration

Angular app registration requests delegated API permissions to:

EXAMPLE / FLOW

```
Fundraising API

Payment API

Reporting API
```

Example:

EXAMPLE / FLOW

```
Fundraising API
→ fundraising.access

Reporting API
→ reporting.read
```

Finance-related users may additionally call Payment API depending on their assigned roles/permissions.

## Roles vs Scopes

Remember:

EXAMPLE / FLOW

```
Scopes
→ what delegated API capability the client/user can access

Roles
→ what organizational/business role the identity has
```

Examples:

EXAMPLE / FLOW

```
Scope:
payments.access
```

and:

EXAMPLE / FLOW

```
Role:
FinanceUser
```

The API may require both.

## App Roles

Possible app roles:

EXAMPLE / FLOW

```
TenantAdministrator

FundraisingManager

Fundraiser

FinanceUser

PaymentOperations

ReportingUser
```

The access token can contain role claims.

ASP.NET Core policies then enforce them.

## API Authorization Flow

EXAMPLE / FLOW

```
Sarah logs in
     |
     v
Entra ID
     |
     v
Token
Scopes:
fundraising.access

Roles:
Fundraiser
     |
     v
POST /api/v1/gifts
     |
     v
Fundraising API
     |
     v
Policy:
CanCreateGift
     |
     v
Allowed
```

## Service-to-Service Identity

Suppose:

EXAMPLE / FLOW

```
Reporting worker
```

needs to access an Azure resource.

Do not give it:

EXAMPLE / FLOW

```
username
password
client secret
```

if Azure Managed Identity can be used.

## Managed Identity

Managed Identity provides an Azure-hosted workload with an Entra identity without requiring you to manage credentials manually.

Examples:

EXAMPLE / FLOW

```
Fundraising Container App
→ managed identity

Payment Container App
→ managed identity

Outbox Worker
→ managed identity

Azure Function
→ managed identity
```

Current Microsoft security guidance recommends managed identity and passwordless authentication where supported.

## System-Assigned Managed Identity

Lifecycle is tied to the Azure resource.

Example:

EXAMPLE / FLOW

```
Fundraising Container App
        |
        v
system-assigned managed identity
```

Delete the Container App and the identity is removed.

Good when:

EXAMPLE / FLOW

```
one workload
→ one identity
```

## User-Assigned Managed Identity

Exists independently.

Can be attached to multiple Azure resources.

Good when several resources need a deliberately shared workload identity.

For this project, start with:

EXAMPLE / FLOW

```
system-assigned managed identities
```

unless shared identity behavior is required.

# AZURE KEY VAULT

## What Key Vault Does

Azure Key Vault protects:

EXAMPLE / FLOW

```
Secrets

Keys

Certificates
```

Examples:

EXAMPLE / FLOW

```
External payment provider secret

Webhook signing secret

certificate

third-party API secret
```

It should not become a generic place for every ordinary configuration value.

## Secret vs Configuration

Non-secret:

EXAMPLE / FLOW

```
PageSize = 50

PaymentTimeoutSeconds = 30

FeatureFlag = true
```

can be normal configuration/environment variables.

Secret:

EXAMPLE / FLOW

```
PaymentProviderApiKey

WebhookSigningSecret

Private certificate password
```

belongs in Key Vault.

## Use Azure RBAC

Azure Key Vault supports legacy access policies and Azure RBAC.

For modern deployments Microsoft recommends the Azure RBAC permission model rather than legacy access policies.

Example:

EXAMPLE / FLOW

```
Fundraising API Managed Identity
        |
        v
Key Vault Secrets User
        |
        v
fp-kv-prod
```

## Key Vault Architecture

EXAMPLE / FLOW

```
Payment Container App
       |
       | Managed Identity
       v
Microsoft Entra ID
       |
       v
Key Vault
       |
       v
PaymentProviderApiKey
```

No hard-coded credential appears in source code.

## Container Apps + Key Vault

Azure Container Apps supports Key Vault-backed secret references.

The Container App managed identity receives permission such as:

EXAMPLE / FLOW

```
Key Vault Secrets User
```

and Container Apps can reference a vault secret rather than requiring you to copy the secret into source control.

## Local Development

Locally, use:

EXAMPLE / FLOW

```
dotnet user-secrets
```

or appropriate developer credentials.

Do not make developers copy production secrets to:

EXAMPLE / FLOW

```
appsettings.Development.json
```

### Production

Use:

EXAMPLE / FLOW

```
Managed Identity
+
Key Vault
```

## ENVIRONMENT VARIABLES AND CONFIGURATION

ASP.NET Core configuration will combine sources.

Conceptually:

EXAMPLE / FLOW

```
appsettings.json
       |
       v
environment-specific settings
       |
       v
environment variables
       |
       v
Key Vault / secret references
```

### appsettings.json

Use it for safe defaults:

JSON

```
{
  "Pagination": {
    "DefaultPageSize": 50,
    "MaximumPageSize": 200
  }
}
```

## Environment Variables

Container Apps can supply:

EXAMPLE / FLOW

```
ASPNETCORE_ENVIRONMENT=Production

PaymentProvider__TimeoutSeconds=30
```

ASP.NET Core translates:

EXAMPLE / FLOW

```
__
```

into nested configuration separators.

Example:

EXAMPLE / FLOW

```
PaymentProvider__TimeoutSeconds
```

maps to:

EXAMPLE / FLOW

```
PaymentProvider:TimeoutSeconds
```

## Do Not Store Secrets in GitHub Environment Variables as Plain Configuration

Prefer:

EXAMPLE / FLOW

```
Managed Identity
Key Vault
OIDC
```

over long-lived cloud credentials.

## Environment-Specific Configuration

Example:

Development:

EXAMPLE / FLOW

```
LogLevel = Debug

Payment Sandbox = true
```

Production:

EXAMPLE / FLOW

```
LogLevel = Information

Payment Sandbox = false
```

Do not create source-code conditions such as:

C#

```
if (Environment == "Production")
{
    providerPassword = "...";
}
```

Use configuration.

## AZURE SQL

## Azure SQL Architecture

Recommended:

EXAMPLE / FLOW

```
Fundraising Service
      |
      v
FundraisingDb

Payment Service
      |
      v
PaymentDb

Reporting Service
      |
      v
ReportingDb
```

For learning/cost control, the three databases can initially live under one Azure SQL logical server.

For stronger production isolation, PaymentDb may later move to its own logical server or more isolated database architecture.

The important rule remains:

EXAMPLE / FLOW

```
each service owns its database.
```

## Passwordless SQL Connection

The preferred Azure design is:

EXAMPLE / FLOW

```
Container App
      |
      v
Managed Identity
      |
      v
Azure SQL
```

rather than:

EXAMPLE / FLOW

```
Connection string
Username
Password
```

Microsoft currently recommends passwordless Azure SQL connections using managed identity where supported.

## SQL User for Managed Identity

Inside FundraisingDb:

SQL

```
CREATE USER [fp-fundraising-api-prod]
FROM EXTERNAL PROVIDER;
```

Then grant your custom role.

Example:

SQL

```
ALTER ROLE FundraisingApiRole
ADD MEMBER [fp-fundraising-api-prod];
```

Better than:

EXAMPLE / FLOW

```
db_owner
```

for the API.

## Payment Identity

Payment API receives its own identity:

EXAMPLE / FLOW

```
fp-payments-api-prod
```

and gets permissions only in:

EXAMPLE / FLOW

```
PaymentDb
```

## Reporting Identity

Reporting API gets:

EXAMPLE / FLOW

```
read-only/reporting access
```

to:

EXAMPLE / FLOW

```
ReportingDb
```

## SQL Connection String

Conceptually:

EXAMPLE / FLOW

```
Server=tcp:<server>.database.windows.net;
Database=FundraisingDb;
Authentication=Active Directory Default;
Encrypt=True;
```

`Microsoft.Data.SqlClient` supports Microsoft Entra and managed-identity authentication for Azure SQL without embedding a SQL password.

## Private Endpoint

For stronger production networking:

EXAMPLE / FLOW

```
Container Apps VNet
      |
      v
Private Endpoint
      |
      v
Azure SQL
```

Then public access to Azure SQL can be disabled.

Azure Private Link/private endpoints provide private-IP connectivity to Azure SQL within your virtual network.

## DOCKER

## Why Docker?

Docker packages:

EXAMPLE / FLOW

```
application
runtime
dependencies
configuration expectations
```

into a container image.

Instead of asking:

EXAMPLE / FLOW

```
"Does this server have the right .NET runtime?"
```

you deploy a known image.

## Containerizing Fundraising API

Create:

EXAMPLE / FLOW

```
src/Fundraising.Api/Dockerfile
```

Typical multi-stage approach:

DOCKERFILE

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build

WORKDIR /src

COPY . .

RUN dotnet restore
RUN dotnet publish \
    src/Fundraising.Api/Fundraising.Api.csproj \
    -c Release \
    -o /app/publish \
    --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime

WORKDIR /app

COPY --from=build /app/publish .

ENTRYPOINT ["dotnet", "Fundraising.Api.dll"]
```

The important idea is:

EXAMPLE / FLOW

```
SDK image
→ build

runtime image
→ run
```

The final image does not need the entire SDK.

## Payment API Image

Separate image:

EXAMPLE / FLOW

```
fundraising-platform/payments-api
```

Do not make one container that runs:

EXAMPLE / FLOW

```
Fundraising API
+
Payment API
+
Reporting API
```

because that destroys independent deployment/scaling.

## Reporting API Image

Separate:

EXAMPLE / FLOW

```
reporting-api
```

## Worker Images

Also separate:

EXAMPLE / FLOW

```
fundraising-outbox-worker

payment-outbox-worker
```

or combine related API/background behavior only if you deliberately accept the lifecycle/scaling coupling.

For learning microservices, separate worker containers make the boundary clearer.

## Docker Compose for Local Development

Use:

EXAMPLE / FLOW

```
docker-compose.yml
```

for local integration.

Example:

EXAMPLE / FLOW

```
Fundraising API
Payment API
Reporting API
SQL Server
Azurite maybe
local messaging emulator/alternative where appropriate
```

Docker Compose is mainly:

EXAMPLE / FLOW

```
local multi-container orchestration
```

not your production Kubernetes replacement.

## Docker Image Tags

Do not use only:

EXAMPLE / FLOW

```
latest
```

Use immutable deployment tags.

Example:

EXAMPLE / FLOW

```
fundraising-api:sha-abc123
```

or:

EXAMPLE / FLOW

```
fundraising-api:1.4.2
```

This gives traceability:

EXAMPLE / FLOW

```
Git commit
→ image
→ Azure deployment
```

## AZURE CONTAINER REGISTRY

### What ACR Does

Azure Container Registry stores private container images.

Flow:

EXAMPLE / FLOW

```
GitHub Actions
      |
      v
docker build
      |
      v
ACR
      |
      v
Container Apps / AKS
```

Images:

EXAMPLE / FLOW

```
fundraising-api
payment-api
reporting-api
outbox-worker
```

## ACR Authentication

Avoid relying on the registry's admin account for production.

Use Microsoft Entra identities/managed identities where possible.

Azure Container Registry supports Entra-based authentication, including managed identities for supported Azure workloads.

## Container App Pull Permission

Container App managed identity needs:

EXAMPLE / FLOW

```
AcrPull
```

on the registry.

GitHub deployment identity may need:

EXAMPLE / FLOW

```
AcrPush
```

or appropriate modern repository-level registry permissions depending on your chosen ACR permission model.

Apply least privilege.

## WHICH AZURE HOSTING SERVICE SHOULD WE USE?

There are three relevant choices:

EXAMPLE / FLOW

```
Azure App Service

Azure Container Apps

Azure Kubernetes Service
```

## Azure App Service

Excellent for:

EXAMPLE / FLOW

```
normal web application

single API

simple HTTP workload
```

Benefits:

EXAMPLE / FLOW

```
very easy deployment

built-in TLS

autoscale

deployment slots

managed identity

VNet integration
```

App Service supports deployment slots in Standard tier and higher, making blue/green-style testing and swapping possible.

## Azure Container Apps

Designed well for:

EXAMPLE / FLOW

```
containerized APIs

microservices

background services

event-driven workers

independent scaling
```

It abstracts the underlying Kubernetes infrastructure.

Current Azure guidance describes Container Apps as a fully managed container platform and a common starting point for teams building containerized microservices without needing direct access to Kubernetes orchestration APIs.

## AKS

Azure Kubernetes Service gives you Kubernetes.

Use when you actually need:

EXAMPLE / FLOW

```
Kubernetes API

custom controllers/operators

complex scheduling

advanced networking

cluster-level control

specialized platform requirements
```

Current AKS offers:

EXAMPLE / FLOW

```
AKS Automatic
```

for a more managed production-ready experience and:

EXAMPLE / FLOW

```
AKS Standard
```

when you need detailed node/cluster configuration. Microsoft currently recommends Automatic for most teams/workloads starting AKS.

## Recommendation for This Project

Use:

EXAMPLE / FLOW

```
Primary implementation
→ Azure Container Apps
```

because you have:

EXAMPLE / FLOW

```
Fundraising API
Payment API
Reporting API
Workers
```

and want:

EXAMPLE / FLOW

```
containers
microservices
scaling
revisions
service discovery
without managing Kubernetes first
```

Then later create:

EXAMPLE / FLOW

```
AKS deployment lab
```

for the same services.

This gives you both skills:

EXAMPLE / FLOW

```
PaaS/serverless containers

Kubernetes
```

without unnecessarily making AKS the first production dependency.

## AZURE CONTAINER APPS

## Container Apps Environment

A Container Apps Environment is a boundary containing related apps.

Example:

EXAMPLE / FLOW

```
fp-prod-env
    |
    +---- fundraising-api
    +---- payments-api
    +---- reporting-api
    +---- fundraising-worker
    +---- payments-worker
```

Container Apps in the same environment get built-in communication/routing capabilities. Azure Container Apps supports name-based service discovery and environment-level load balancing between apps.

## External vs Internal Ingress

Angular/public clients need:

EXAMPLE / FLOW

```
external ingress
```

only on the controlled public entry layer.

Your internal services can use:

EXAMPLE / FLOW

```
internal ingress
```

where appropriate.

Example:

EXAMPLE / FLOW

```
API Management
      |
      v
Fundraising API
```

Payment internal worker should not necessarily be directly public.

Azure Container Apps supports external and internal ingress modes.

## Recommended Exposure

Production-style:

EXAMPLE / FLOW

```
Front Door
     |
     v
API Management
     |
     v
APIs
```

The APIs themselves should not need unrelated public endpoints exposed directly if your network architecture routes them privately.

## Container App Revisions

Every deploy can produce a new immutable revision.

Example:

EXAMPLE / FLOW

```
fundraising-api--rev1

fundraising-api--rev2
```

Container Apps supports revision management, traffic splitting, and rollback-oriented deployment strategies.

## Zero-Downtime Style Deployment

Single revision mode:

EXAMPLE / FLOW

```
Revision 1 handles requests
        |
        v
Deploy Revision 2
        |
        v
Revision 2 becomes healthy
        |
        v
traffic switches
```

The existing revision remains active until the new revision is ready in the normal single-revision deployment behavior.

## Blue/Green or Canary

Multiple revisions allow:

EXAMPLE / FLOW

```
Revision A
90%

Revision B
10%
```

Then:

EXAMPLE / FLOW

```
monitor B
```

If healthy:

EXAMPLE / FLOW

```
B = 100%
```

If unhealthy:

EXAMPLE / FLOW

```
A = 100%
```

Container Apps supports weighted traffic splitting between active revisions for these scenarios.

## Autoscaling

Fundraising API can scale based on:

EXAMPLE / FLOW

```
HTTP concurrency
```

Payment worker can scale based on:

EXAMPLE / FLOW

```
Service Bus queue depth
```

Container Apps uses KEDA-based autoscaling and supports HTTP, CPU/memory, and event-driven metrics including Azure Service Bus.

## Example API Scaling

EXAMPLE / FLOW

```
min replicas = 2

max replicas = 20
```

Why production minimum > 0?

Because an important API may need:

EXAMPLE / FLOW

```
consistent low latency
```

instead of cold-start behavior.

## Example Worker Scaling

EXAMPLE / FLOW

```
Service Bus queue
0 messages
→ 0/low workers

10,000 messages
→ scale workers up
```

Do not configure unlimited parallel consumers without considering:

EXAMPLE / FLOW

```
SQL connection capacity
payment provider rate limits
downstream load
```

## CONTAINER APPS JOBS

Container Apps Jobs are useful for finite work that:

EXAMPLE / FLOW

```
starts
runs
finishes
```

They can be:

EXAMPLE / FLOW

```
manual

scheduled

event-triggered
```

Current Container Apps Jobs support all three modes.

## Good Project Jobs

Examples:

EXAMPLE / FLOW

```
nightly reporting ETL

historical data cleanup

one-time migration

monthly reconciliation job

large report generation
```

## Do Not Use a Job for the Normal API

API:

EXAMPLE / FLOW

```
long-running service
→ Container App
```

Batch:

EXAMPLE / FLOW

```
finite execution
→ Container Apps Job
```

# AZURE FUNCTIONS

## What Azure Functions Are

Functions provide event-driven serverless execution.

Good examples:

EXAMPLE / FLOW

```
Service Bus message handler

scheduled cleanup

small integration

webhook processing

file processing
```

Azure Functions is designed around event-driven triggers and bindings.

## Recommended .NET Functions Model

Use:

EXAMPLE / FLOW

```
.NET isolated worker
```

for new .NET Functions development.

Microsoft has announced the end of support for the in-process Functions model on November 10, 2026 and recommends migration to isolated worker.

## Functions Hosting

For serverless Functions, current Azure guidance recommends:

EXAMPLE / FLOW

```
Flex Consumption
```

as the recommended serverless hosting plan, supporting features including private networking and configurable scaling.

## Where Functions Fit in Our Project

Do not rewrite the entire API as Functions.

Use Functions for focused event-driven tasks.

Examples:

EXAMPLE / FLOW

```
Payment webhook normalization

daily reporting ETL trigger

Service Bus integration processor

document/file processing

scheduled cleanup
```

## Function vs Container Worker

Use a Function when:

EXAMPLE / FLOW

```
work is event-triggered

execution is discrete

serverless scaling is useful
```

Use a Container App worker when:

EXAMPLE / FLOW

```
long-running process

continuous Service Bus consumer

complex custom runtime

shared container deployment model
```

Both approaches are valid.

# AZURE SERVICE BUS

## Why Service Bus?

Fundraising and Payment microservices must not share database transactions.

Use events.

EXAMPLE / FLOW

```
Fundraising
     |
     v
GiftCreated
     |
     v
Service Bus
     |
     v
Payment
```

Azure Service Bus provides reliable queues and publish/subscribe messaging.

## Queue

A queue is:

EXAMPLE / FLOW

```
one message
→ one competing consumer
```

Example:

EXAMPLE / FLOW

```
payment-processing queue
```

Several Payment workers may consume, but each message is processed by one consumer.

## Topic + Subscription

Topic:

EXAMPLE / FLOW

```
one event
→ many interested subscribers
```

Example:

EXAMPLE / FLOW

```
GiftCreated topic
```

Subscriptions:

EXAMPLE / FLOW

```
Payment subscription

Reporting subscription

Notification subscription
```

Flow:

EXAMPLE / FLOW

```
GiftCreated
     |
     v
Topic
 |
 +---- Payment subscription
 |
 +---- Reporting subscription
 |
 +---- Notification subscription
```

For our event-driven microservices, Topics are extremely useful.

## Recommended Topics

Potential:

EXAMPLE / FLOW

```
fundraising-events

payment-events
```

or more domain-specific arrangements.

Events:

EXAMPLE / FLOW

```
GiftCreated

GiftUpdated

PaymentSucceeded

PaymentFailed

RefundCompleted

SettlementCompleted
```

## Dead-Letter Queue

Messages that cannot be successfully processed after retry should eventually move to a dead-letter queue.

Flow:

EXAMPLE / FLOW

```
Message
   |
   v
Consumer
   |
 fails
   |
 retry
   |
 fails repeatedly
   |
   v
DLQ
```

Operations need monitoring for:

EXAMPLE / FLOW

```
dead-letter message count
```

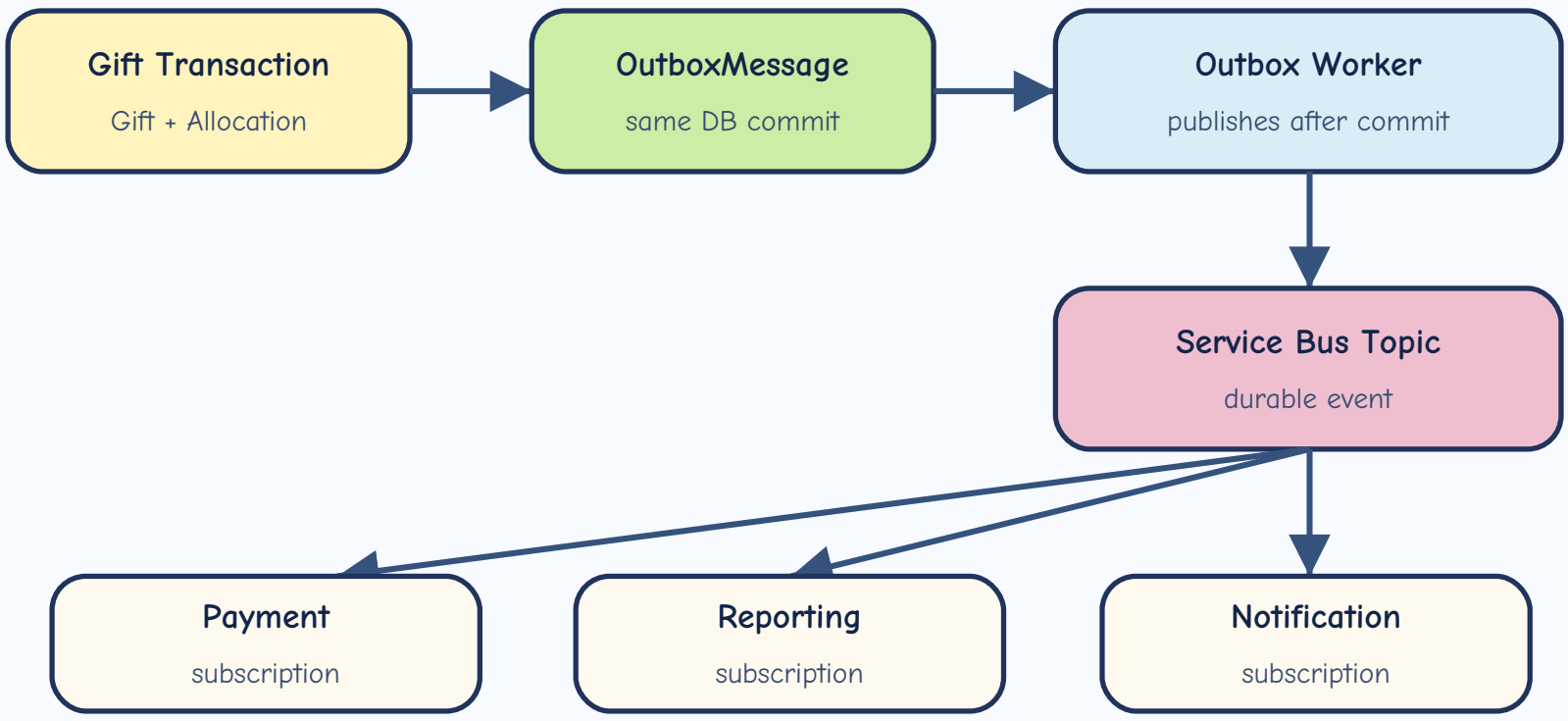
Do not allow DLQs to become invisible graveyards.

## Outbox + Service Bus

Correct flow:

DIAGRAM

Outbox + Service Bus — Reliable Cross-Service Flow




The fundraising transaction commits its event locally; a worker publishes it later so Payment and Reporting remain independent.

EXAMPLE / FLOW

```
Fundraising transaction
        |
        +---- Gift
        +---- GiftAllocation
        +---- OutboxMessage
        |
        v
COMMIT
        |
        v
Outbox Worker
        |
        v
Service Bus Topic
        |
        v
Payment Consumer
```

This preserves service/database independence.

## Managed Identity + Service Bus

Azure workloads should authenticate to Service Bus using managed identity/RBAC instead of hard-coded connection strings where practical.

Example roles:

EXAMPLE / FLOW

```
Azure Service Bus Data Sender

Azure Service Bus Data Receiver
```

Grant only what each worker needs.

## API MANAGEMENT

## Why API Management?

Without APIM:

EXAMPLE / FLOW

```
Angular
   |
   +---- fundraising-api.azurecontainerapps.io
   +---- payment-api.azurecontainerapps.io
   +---- reporting-api.azurecontainerapps.io
```

Angular now knows all your internal services.

With APIM:

EXAMPLE / FLOW

```
Angular
    |
    v
api.example.com
    |
    v
API Management
    |
    +---- /fundraising
    +---- /payments
    +---- /reports
```

Azure API Management is designed to publish and protect backend APIs through a managed gateway.

## APIM Responsibilities

Use APIM for:

EXAMPLE / FLOW

```
routing

API exposure

rate limiting

request policies

header transformations

API version exposure

monitoring

JWT/Entra token validation

subscription/product management where needed
```

Do not move all business logic into APIM policy XML.

## Import OpenAPI

Your ASP.NET APIs already generate OpenAPI.

You can import those specifications into API Management. Azure APIM supports importing OpenAPI definitions.

Flow:

EXAMPLE / FLOW

```
ASP.NET OpenAPI
      |
      v
API Management import
      |
      v
published API
```

## Validate Entra Token at APIM

API Management can validate Microsoft Entra tokens using:

EXAMPLE / FLOW

```
validate-azure-ad-token
```

policy.

But the backend API should still enforce its authorization.

Think:

EXAMPLE / FLOW

```
APIM
→ edge validation

API
→ actual application authorization
```

Defense in depth.

## AZURE FRONT DOOR

### Why Front Door?

API Management is an API gateway.

Front Door is a global edge/application-delivery service.

It gives:

EXAMPLE / FLOW

```
global edge routing

TLS

CDN capabilities

Web Application Firewall

origin protection

traffic acceleration
```

Azure Front Door provides global edge delivery, routing, security capabilities, and WAF integration.

### Architecture

EXAMPLE / FLOW

```
User
 |
 v
Front Door
 |
 +---- Angular
 |
 +---- APIM
```

### WAF

Web Application Firewall protects against classes of malicious HTTP traffic.

Use managed rules plus carefully tested custom policies.

Do not think:

EXAMPLE / FLOW

```
WAF means API authorization is unnecessary.
```

It solves a different security problem.

## Front Door Classic

Do not start new architecture on:

EXAMPLE / FLOW

```
Azure Front Door Classic
```

Microsoft lists its retirement for March 31, 2027; use Standard/Premium for modern designs.

## LOAD BALANCING

There are multiple load-balancing levels.

Do not think Azure has one single load balancer for everything.

## Container Apps Load Distribution

Container Apps automatically distributes traffic across replicas/revisions according to its ingress/revision configuration.

## Front Door

Global:

EXAMPLE / FLOW

```
User
→ best edge/origin
```

## Azure Load Balancer

Layer-4 TCP/UDP service.

More relevant for lower-level VM/network architectures.

You probably do not need to manually place Azure Load Balancer in front of your Container Apps APIs.

## Application Gateway

Regional Layer-7 gateway with WAF.

Useful in architectures requiring regional/private ingress.

Do not automatically deploy:

EXAMPLE / FLOW

```
Front Door
+
Application Gateway
+
APIM
+
Container Apps ingress
```

unless each layer has a reason.

## NETWORKING Virtual Network

A VNet gives you a private network boundary.

Production architecture may include:

EXAMPLE / FLOW

```
VNet
 |
 +---- Container Apps subnet
 |
 +---- Private Endpoint subnet
```

## Private Endpoints

Use Private Link for services such as:

EXAMPLE / FLOW

```
Azure SQL

Key Vault

Storage

Container Apps environment when required
```

Azure Container Apps also supports private endpoints for private inbound connectivity when using supported environment configurations.

## Private DNS

Private endpoints require DNS to resolve service hostnames to private IPs.

For example:

EXAMPLE / FLOW

```
database.windows.net
```

needs correct private-DNS behavior after private endpoint integration.

Networking failures are frequently actually:

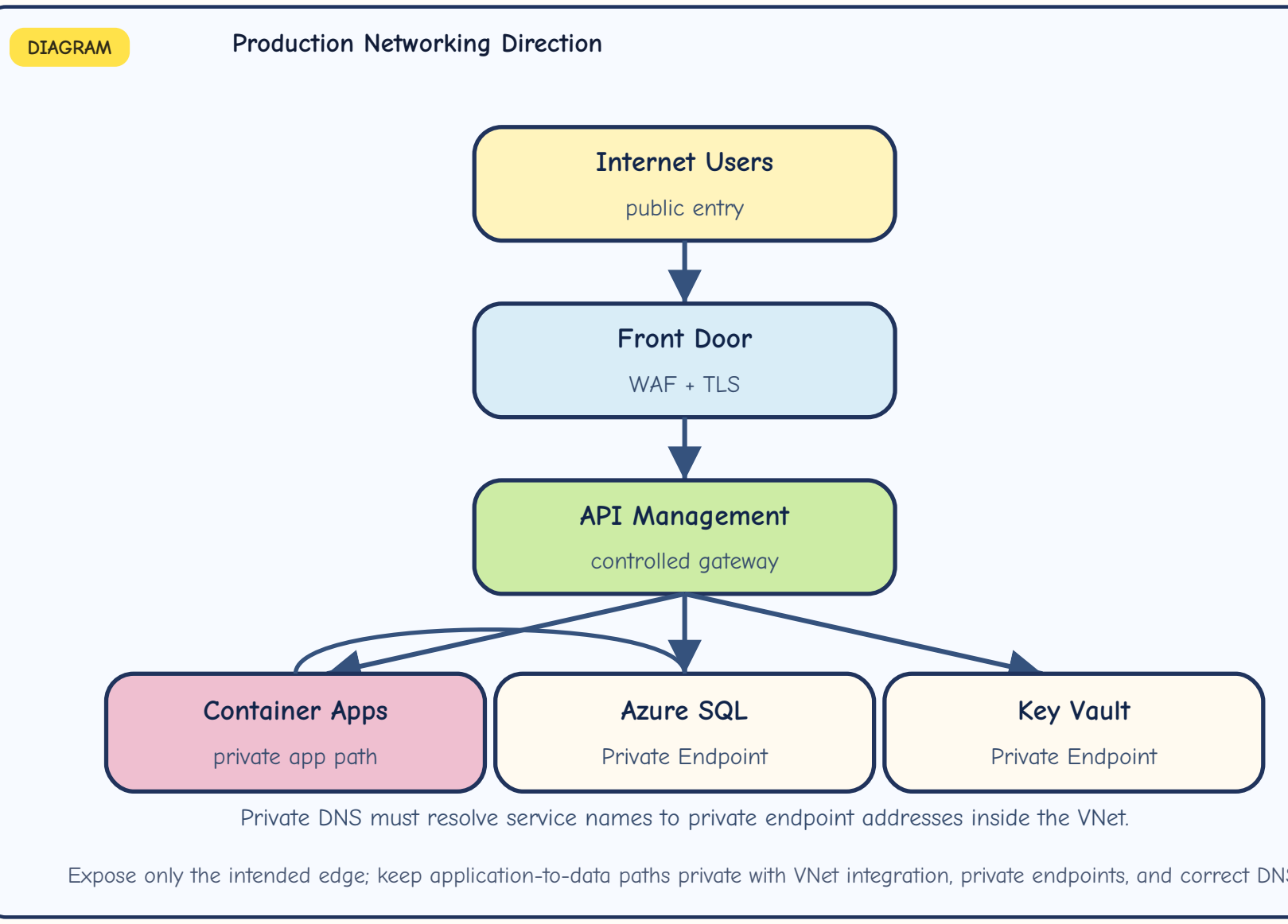
EXAMPLE / FLOW

```
DNS failures.
```

## Production Networking Direction

Conceptually:

DIAGRAM




Private DNS must resolve service names to private endpoint addresses inside the VNet.

Expose only the intended edge; keep application-to-data paths private with VNet integration, private endpoints, and correct DNS.

EXAMPLE / FLOW

```
Internet
    |
    v
Front Door
    |
    v
APIM
    |
    v
Private application network
    |
    +---- Fundraising API
    +---- Payment API
    +---- Reporting API
         |
         v
Private endpoints
         |
    +---- Azure SQL
    +---- Key Vault
```

Build this after the simpler deployment works.

Do not begin your first Azure learning deployment with maximum private-network complexity.

## GITHUB Repository

Recommended monorepo initially:

EXAMPLE / FLOW

```
fundraising-platform/
│
├── src/
│   ├── Fundraising.*
│   ├── Payments.*
│   └── Reporting.*
│
├── tests/
│
├── angular/
│
├── infra/
│   └── bicep/
│
├── .github/
│   └── workflows/
│
├── docker-compose.yml
└── README.md
```

A monorepo is reasonable while learning.

Microservice does not necessarily mean:

EXAMPLE / FLOW

```
one Git repository per service.
```

## Branching

Simple approach:

EXAMPLE / FLOW

```
main
```

is deployable.

Feature work:

EXAMPLE / FLOW

```
feature/create-gift
```

Workflow:

EXAMPLE / FLOW

```
feature branch
     |
     v
Pull Request
     |
     v
Build + Test
     |
     v
Code Review
     |
     v
Merge
```

Avoid maintaining complicated long-lived Gitflow branches unless your organization requires them.

## Pull Requests

PR checks should include:

EXAMPLE / FLOW

```
dotnet restore

dotnet build

unit tests

integration tests where practical

Angular build/tests

format/lint checks

security scanning

container build
```

No direct production deployment from an unreviewed developer branch.

## GITHUB ACTIONS

## CI vs CD

CI:

EXAMPLE / FLOW

```
Continuous Integration
```

means:

EXAMPLE / FLOW

```
build
test
validate
```

whenever code changes.

CD:

EXAMPLE / FLOW

```
Continuous Delivery/Deployment
```

means promoting successful code to environments.

## CI Flow

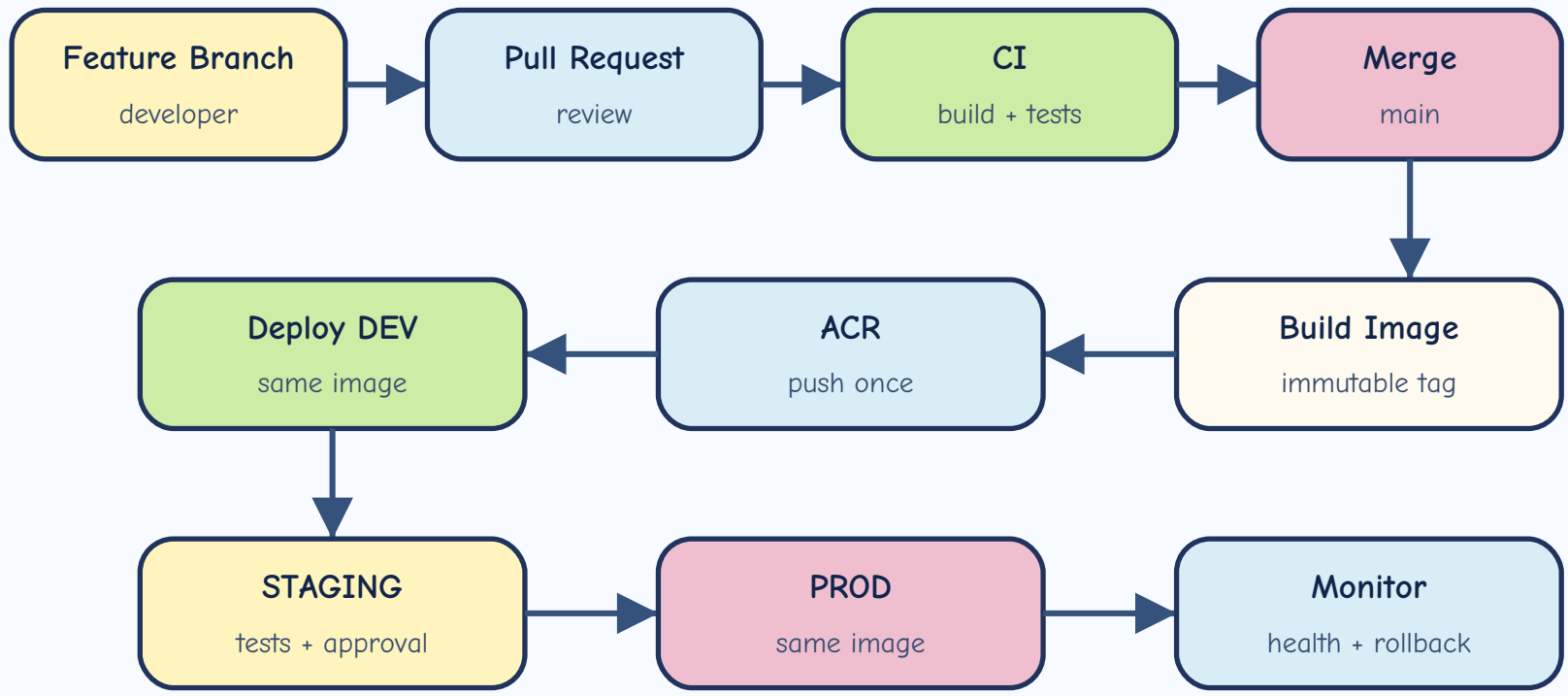
EXAMPLE / FLOW

```
git push
   |
   v
GitHub Actions
   |
   +---- restore
   |
   +---- build
   |
   +---- unit tests
   |
   +---- integration tests
   |
   +---- Docker build
```

## CD Flow

DIAGRAM

GitHub Actions — Build Once, Promote Safely




Build once → promote the exact tested artifact → monitor → roll back by revision/image when required.

CI validates the code; CD publishes one immutable image and promotes it through Dev, Staging, and Production.

EXAMPLE / FLOW

```
Merge to main
      |
      v
Build image
      |
      v
Push image to ACR
      |
      v
Deploy Dev
      |
      v
Automated smoke tests
      |
      v
Promote Staging
      |
      v
Approval
      |
      v
Promote Production
```

## GitHub Authentication to Azure

Do not store a long-lived:

EXAMPLE / FLOW

```
Azure username/password
```

or broad service-principal secret if it can be avoided.

Use:

EXAMPLE / FLOW

```
OpenID Connect
+
federated identity credential
```

GitHub Actions can authenticate to Azure using OIDC rather than a long-lived deployment password. Current Microsoft Functions/GitHub guidance explicitly recommends OIDC for deployment authentication.

## GitHub Federated Identity Flow

EXAMPLE / FLOW

```
GitHub Actions
       |
       | OIDC token
       v
Microsoft Entra ID
       |
       v
Federated Identity Credential
       |
       v
Azure deployment identity
       |
       v
Azure RBAC
```

No long-lived Azure password needs to be stored in GitHub.

## Deployment Identity Permissions

Do not grant:

EXAMPLE / FLOW

```
Owner
```

at entire subscription if not required.

Give deployment identity access to:

EXAMPLE / FLOW

```
specific resource group
or
required resources
```

with appropriate roles.

## Example GitHub Workflow — CI

YAML

```yaml
name: Fundraising API CI

on:
  pull_request:
    paths:
      - "src/Fundraising.**"
      - "tests/Fundraising.**"

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "10.0.x"

      - run: dotnet restore

      - run: dotnet build --configuration Release --no-restore

      - run: dotnet test --configuration Release --no-build
```

The exact action versions should be kept updated in your actual repository.

## Deployment Workflow

Conceptually:

YAML

```yaml
permissions:
  id-token: write
  contents: read
```

Then:

EXAMPLE / FLOW

```
azure/login
```

using:

EXAMPLE / FLOW

```
client ID
tenant ID
subscription ID
```

to perform OIDC-based login.

## Container Apps GitHub Deployment

Azure Container Apps supports GitHub Actions-based deployment, where a commit can build/push a container and create a new Container App revision.

Recommended production flow:

EXAMPLE / FLOW

```
GitHub Actions
      |
      v
Build Docker image
      |
      v
Run container/security checks
      |
      v
Push immutable tag to ACR
      |
      v
Deploy that image to Container Apps
```

## Build Once, Promote Same Image

Do not rebuild different binaries for:

EXAMPLE / FLOW

```
Dev

Staging

Prod
```

Prefer:

EXAMPLE / FLOW

```
one tested image
```

promoted through environments.

Example:

EXAMPLE / FLOW

```
fundraising-api:sha-c71ab4
```

deploy:

EXAMPLE / FLOW

```
Dev
→ same image

Staging
→ same image

Prod
→ same image
```

Only configuration changes.

## Environment Protection

Use GitHub Environments:

EXAMPLE / FLOW

```
development

staging

production
```

Production environment can require:

EXAMPLE / FLOW

```
manual approval
```

before deployment.

## INFRASTRUCTURE AS CODE

Do not create all production resources manually through the portal forever.

Use:

EXAMPLE / FLOW

```
Bicep
```

or:

EXAMPLE / FLOW

```
Terraform
```

## Recommendation

For an Azure-focused Microsoft stack learning project:

EXAMPLE / FLOW

```
Bicep
```

is a strong first choice.

Then learn Terraform if you want cloud-neutral IaC skills.

## Bicep Repository Structure

EXAMPLE / FLOW

```
infra/
│
├── modules/
│   ├── container-app.bicep
│   ├── sql.bicep
│   ├── keyvault.bicep
│   ├── servicebus.bicep
│   ├── acr.bicep
│   └── monitoring.bicep
│
├── environments/
│   ├── dev.bicepparam
│   ├── staging.bicepparam
│   └── prod.bicepparam
│
└── main.bicep
```

## Infrastructure Pipeline

EXAMPLE / FLOW

```
Pull Request
     |
     v
Bicep validation
     |
     v
What-if
     |
     v
review
     |
     v
deploy
```

Do not blindly execute infrastructure changes in production.

## Azure Developer CLI

You can also learn:

EXAMPLE / FLOW

```
azd
```

for local-to-Azure developer workflows.

It can provision infrastructure and integrate deployment workflows.

Use it as a developer productivity tool, while still understanding the Bicep resources underneath.

## DEPLOYING EACH MICROSERVICE

## Fundraising Service Deployment

Resources:

EXAMPLE / FLOW

```
Container App:
fp-fundraising-api-prod

Managed Identity:
system assigned

Database:
FundraisingDb

Key Vault access:
required Fundraising secrets only

Service Bus:
sender for fundraising events

ACR:
AcrPull

Application Insights:
enabled
```

Ingress:

EXAMPLE / FLOW

```
reachable through APIM
```

## Fundraising Worker

EXAMPLE / FLOW

```
fp-fundraising-worker-prod
```

Responsibilities:

EXAMPLE / FLOW

```
read OutboxMessage

publish Service Bus events

mark processed
```

Scaling:

EXAMPLE / FLOW

```
low/min replicas
```

or event/worker-specific configuration.

## Payment Service Deployment

EXAMPLE / FLOW

```
fp-payments-api-prod
```

Access:

EXAMPLE / FLOW

```
PaymentDb

Payment Key Vault secrets

Service Bus

external payment provider
```

Payment secrets should not be visible to Fundraising Service.

## Payment Worker

Consumes:

EXAMPLE / FLOW

```
GiftCreated
```

Creates:

EXAMPLE / FLOW

```
Payment
```

Publishes:

EXAMPLE / FLOW

```
PaymentSucceeded

PaymentFailed
```

## Reporting Service

EXAMPLE / FLOW

```
fp-reporting-api-prod
```

Access:

EXAMPLE / FLOW

```
ReportingDb only
```

Normal Angular report queries should not require direct FundraisingDb or PaymentDb access.

## Reporting ETL

Can run as:

EXAMPLE / FLOW

```
Container Apps Job

Azure Function

continuous worker
```

depending on update requirements.

Example scheduled batch:

EXAMPLE / FLOW

```
nightly ETL
→ Container Apps scheduled job
```

Near-real-time:

EXAMPLE / FLOW

```
Service Bus event consumer
→ worker / Function
```

## ANGULAR DEPLOYMENT

Angular is a static SPA after production build.

Options:

EXAMPLE / FLOW

```
Azure Static Web Apps

Azure Storage static website + Front Door

App Service
```

For a normal Angular SPA:

EXAMPLE / FLOW

```
Azure Static Web Apps
```

is a simple fit.

For an architecture where Front Door is already the central global entry:

EXAMPLE / FLOW

```
Storage/static hosting
+
Front Door
```

can also be appropriate.

## Angular Production Build

Pipeline:

EXAMPLE / FLOW

```
npm ci

npm test

npm run build
```

Output:

EXAMPLE / FLOW

```
dist/
```

Deploy static assets.

## Angular Configuration

Do not compile:

EXAMPLE / FLOW

```
production secret
```

into Angular.

Anything shipped to a browser can be inspected.

Safe browser configuration:

EXAMPLE / FLOW

```
API base URL

Entra client ID

tenant ID

scope names
```

Secret:

EXAMPLE / FLOW

```
Payment provider private secret
```

must never be placed in Angular.

## KUBERNETES

You specifically want Kubernetes.

We should learn it, but use it intentionally.

## Kubernetes Architecture

EXAMPLE / FLOW

```
AKS Cluster
    |
    +---- Namespace
    |
    +---- Deployment
    |
    +---- Pods
    |
    +---- Services
    |
    +---- Ingress
```

## Pod

Smallest deployable Kubernetes unit.

Example:

EXAMPLE / FLOW

```
fundraising-api pod
```

contains:

EXAMPLE / FLOW

```
Fundraising API container
```

## Deployment

A Deployment manages desired pod replicas.

Example:

YAML

```
replicas: 3
```

Kubernetes tries to maintain three Fundraising API pods.

## Service

Pods are temporary.

A Kubernetes Service provides a stable network endpoint.

EXAMPLE / FLOW

```
fundraising-api-service
       |
       +---- Pod 1
       +---- Pod 2
       +---- Pod 3
```

## Ingress

Ingress provides HTTP routing into the cluster through an ingress controller.

Example:

EXAMPLE / FLOW

```
/api/fundraising
→ Fundraising Service

/api/payments
→ Payment Service
```

In Azure you might still place:

EXAMPLE / FLOW

```
Front Door
+
API Management
```

outside AKS depending on architecture.

## ConfigMap

Non-secret Kubernetes configuration.

Example:

EXAMPLE / FLOW

```
PaymentTimeoutSeconds=30
```

## Kubernetes Secret

Can store sensitive data, but for Azure architecture prefer:

EXAMPLE / FLOW

```
Key Vault
+
Workload Identity
```

for important secrets instead of manually maintaining raw Kubernetes secrets where possible.

## AKS Workload Identity

Workload Identity allows Kubernetes workloads to authenticate to Azure resources through Microsoft Entra federation.

Example:

EXAMPLE / FLOW

```
Payment pod
      |
      v
Service Account
      |
      v
Workload Identity
      |
      v
Key Vault / Service Bus / Azure SQL
```

This avoids embedding Azure credentials into pod configuration.

## AKS Autoscaling

Learn:

EXAMPLE / FLOW

```
Horizontal Pod Autoscaler

Cluster autoscaler / automatic node management

KEDA
```

Depending on AKS mode/configuration.

## Kubernetes Health Probes

Your ASP.NET APIs already expose:

EXAMPLE / FLOW

```
/health/live

/health/ready
```

Kubernetes maps those to:

EXAMPLE / FLOW

```
livenessProbe

readinessProbe
```

If readiness fails:

EXAMPLE / FLOW

```
stop routing new requests
```

If liveness repeatedly fails:

EXAMPLE / FLOW

```
restart container
```

## AKS Deployment Path

Do not begin with AKS before you understand your services.

Sequence:

EXAMPLE / FLOW

```
Docker locally
      |
      v
ACR
      |
      v
Container Apps
      |
      v
learn working cloud system
      |
      v
AKS
      |
      v
deploy same containers
```

This separates:

EXAMPLE / FLOW

```
application problems
```

from:

EXAMPLE / FLOW

```
Kubernetes problems.
```

## AKS Automatic vs Standard

Current AKS offers:

EXAMPLE / FLOW

```
Automatic
```

with more Azure-managed operational defaults, and:

EXAMPLE / FLOW

```
Standard
```

for deeper cluster/node control.

For your first AKS deployment:

EXAMPLE / FLOW

```
AKS Automatic
```

is a sensible learning starting point.

Then investigate Standard when learning:

EXAMPLE / FLOW

```
node pools

special VM SKUs

advanced networking

custom infrastructure
```

## CI/CD ENVIRONMENT FLOW

Recommended pipeline:

EXAMPLE / FLOW

```
Feature Branch
      |
      v
Pull Request
      |
      +---- .NET build
      +---- xUnit
      +---- Angular tests
      +---- lint
      +---- Bicep validation
      +---- Docker build
      |
      v
Merge main
      |
      v
Build immutable artifacts
      |
      +---- Fundraising image
      +---- Payment image
      +---- Reporting image
      +---- Worker images
      +---- Angular build
      |
      v
```

EXAMPLE / FLOW — continued

```
Push images to ACR
      |
      v
Deploy DEV
      |
      v
Integration tests
      |
      v
Deploy STAGING
      |
      v
Smoke tests
      |
      v
Approval
      |
      v
Deploy PROD
      |
      v
Health checks
      |
      v
Monitor
```

## Database Migration Pipeline

Database migrations require extra care.

Do not let every running API replica independently attempt major schema migration at startup.

Better:

EXAMPLE / FLOW

```
Deploy pipeline
      |
      v
database migration step/job
      |
      v
verify
      |
      v
deploy application revision
```

For zero-downtime deployments use:

EXAMPLE / FLOW

```
backward-compatible schema changes
```

when possible.

## Expand-and-Contract Pattern

Example:

Old code uses:

EXAMPLE / FLOW

```
PrimaryEmail
```

New model needs:

EXAMPLE / FLOW

```
EmailAddress
```

Do not immediately:

EXAMPLE / FLOW

```
rename column
deploy app
```

if old instances are still running.

Use:

EXAMPLE / FLOW

```
1. Add new structure
2. Deploy compatible code
3. migrate data
4. switch reads/writes
5. later remove old structure
```

This is important with blue/green/revision deployments.

## DEPLOYMENT STRATEGIES

### Rolling Deployment

Replace replicas gradually.

Common platform behavior:

### Blue/Green

EXAMPLE / FLOW

```
Blue
current production

Green
new release
```

Test Green.

Then switch traffic.

Container Apps revision labels/traffic management and App Service slots can support blue/green-style deployment approaches.

### Canary Deployment

EXAMPLE / FLOW

```
Version 1
95%

Version 2
5%
```

Observe:

EXAMPLE / FLOW

```
errors

latency

business metrics
```

Then increase.

Container Apps multiple revision mode supports percentage-based traffic splitting.

## Rollback

If new revision fails:

EXAMPLE / FLOW

```
move traffic to previous healthy revision.
```

Image tags and revisions make rollback traceable.

## APPLICATION INSIGHTS / AZURE MONITOR

## Observability

You need three categories:

EXAMPLE / FLOW

```
Logs

Metrics

Traces
```

## Logs

Example:

EXAMPLE / FLOW

```
GiftCreated
PaymentAttemptFailed
WebhookProcessingFailed
```

Use structured logging.

## Metrics

Examples:

EXAMPLE / FLOW

```
request rate

HTTP 500 rate

API latency

SQL dependency duration

queue length

payment success rate

worker failure rate
```

## Distributed Tracing

One Gift may travel:

EXAMPLE / FLOW

```
Angular
   |
   v
Fundraising API
   |
   v
Service Bus
   |
   v
Payment Worker
   |
   v
Payment Provider
```

Distributed tracing lets you follow that operation.

## OpenTelemetry

Use OpenTelemetry instrumentation in your .NET services.

Export telemetry to:

EXAMPLE / FLOW

```
Azure Monitor / Application Insights
```

Microsoft provides Azure Monitor/OpenTelemetry integration for .NET applications and Application Insights acts as Azure's APM experience.

## What to Monitor

Fundraising:

EXAMPLE / FLOW

```
POST Gift latency

Gift failures

Outbox backlog
```

Payments:

EXAMPLE / FLOW

```
provider response time

failure rate

refund failures

webhook backlog
```

Reporting:

EXAMPLE / FLOW

```
report query duration

cache hit rate

ETL duration
```

Infrastructure:

EXAMPLE / FLOW

```
CPU

memory

replicas

HTTP concurrency

Service Bus backlog

SQL utilization
```

## Alerts

Create alerts such as:

EXAMPLE / FLOW

```
HTTP 5xx > threshold

Payment failures spike

DLQ messages > 0

Outbox backlog growing

SQL utilization high

Container repeatedly restarting

Key Vault access failures
```

An alert must lead to an actionable operational response.

## HEALTH CHECKS

Fundraising:

EXAMPLE / FLOW

```
/health/live

/health/ready
```

Readiness may verify important dependencies carefully.

Do not make it run:

EXAMPLE / FLOW

```
large SQL query
```

every few seconds.

## Dependency Health

Examples:

EXAMPLE / FLOW

```
SQL availability

Service Bus availability
```

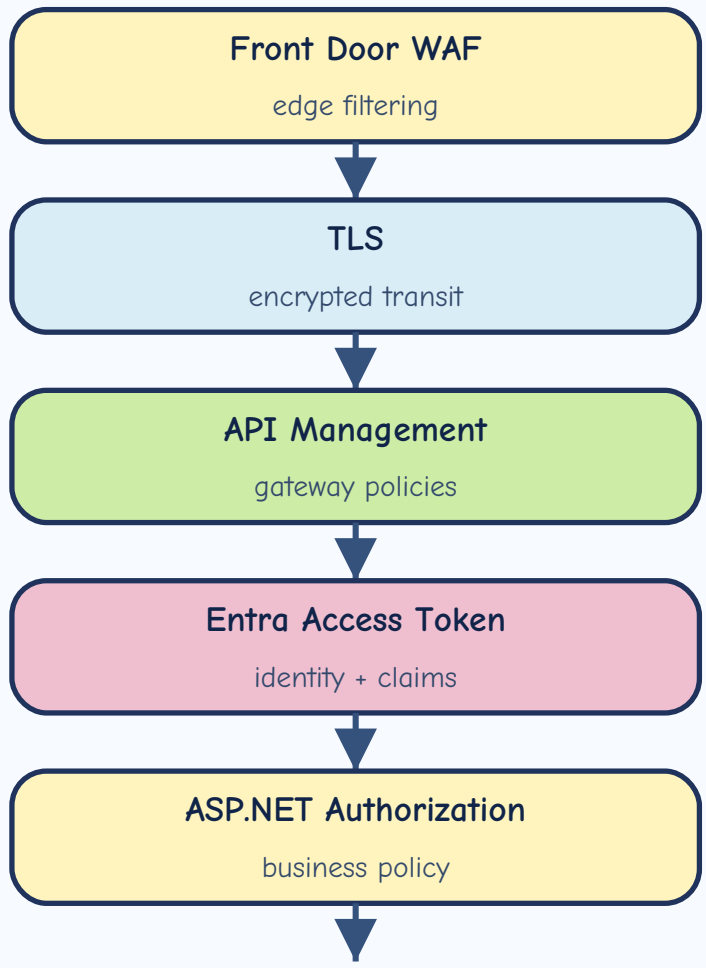
External payment provider availability is often better represented separately than making your entire API “unready” every time the provider has a brief issue.

## SECURITY ARCHITECTURE

Use layers.

DIAGRAM

Defense in Depth — Azure Security Layers




Managed Identity → Azure SQL / Key Vault / Service Bus → least privilege

Each layer solves a different threat: edge protection, encrypted transport, API validation, application authorization, workload identity, and least-privilege data access.

EXAMPLE / FLOW

```
Front Door WAF
      |
      v
TLS
      |
      v
API Management
      |
      v
Entra Access Token
      |
      v
ASP.NET Authorization
      |
      v
Managed Identity
      |
      v
Azure SQL / Key Vault / Service Bus
      |
      v
Database least privilege
```

## No Long-Lived Secrets Where Identity Works

Preferred:

EXAMPLE / FLOW

```
Managed Identity
```

instead of:

EXAMPLE / FLOW

```
SQL password

Service Bus connection string

Key Vault secret for another Azure service
```

when the Azure service supports Entra authentication.

## RBAC

Azure RBAC controls:

EXAMPLE / FLOW

```
who can manage/use Azure resources.
```

Examples:

EXAMPLE / FLOW

```
Key Vault Secrets User

AcrPull

Service Bus Data Sender

Service Bus Data Receiver
```

Assign the minimum required scope.

## Different Identity per Service

Good:

EXAMPLE / FLOW

```
Fundraising API identity

Payment API identity

Reporting API identity
```

Bad:

EXAMPLE / FLOW

```
One giant PlatformIdentity
with access to everything
```

If Payment is compromised, it should not automatically have unrelated Fundraising administration permissions.

## HIGH AVAILABILITY AND DISASTER RECOVERY

Your Azure deployment must consider:

EXAMPLE / FLOW

```
Availability

Recovery

Backup

regional failure
```

## Azure SQL

Use Azure SQL capabilities for:

EXAMPLE / FLOW

```
automated backups

point-in-time recovery

high availability
```

But still understand:

EXAMPLE / FLOW

```
RPO

RTO

regional DR requirements
```

from your SQL Server studies.

## Application Tier

Container Apps can run multiple replicas.

Avoid:

EXAMPLE / FLOW

```
one process
with critical in-memory state
```

because another replica must be able to process the next request.

Keep APIs:

EXAMPLE / FLOW

```
stateless
```

where possible.

## Service Bus

Messaging decouples services during short outages.

Example:

EXAMPLE / FLOW

```
Payment Service down
```

does not necessarily mean:

EXAMPLE / FLOW

```
Gift creation must fail.
```

Gift event remains queued.

Payment resumes later.

That is one of the strongest benefits of asynchronous architecture.

## Regional Disaster Recovery

Advanced production architecture might use:

EXAMPLE / FLOW

```
Primary Azure region

Secondary Azure region
```

with:

EXAMPLE / FLOW

```
Front Door
```

routing during regional failure.

Do this after mastering single-region deployment.

## COST MANAGEMENT

Do not create expensive resources blindly.

For learning:

EXAMPLE / FLOW

```
Dev resources
→ lower SKUs

scale-to-zero where appropriate

delete temporary labs
```

Useful tools:

EXAMPLE / FLOW

```
Azure Cost Management

Budgets

Alerts

Tags
```

Tag resources:

EXAMPLE / FLOW

```
Environment=Dev

Project=FundraisingPlatform

Owner=...
```

## Production Cost Rule

Do not choose the smallest SKU purely because it is cheap.

Do not choose the largest because it "must be faster."

Measure:

EXAMPLE / FLOW

```
CPU

memory

latency

SQL utilization

queue backlog
```

then size appropriately.

## WHAT TO DEPLOY FIRST

Do not create the final architecture in one day.

Use this sequence.

## Azure Stage 1 — Identity Foundation

Create:

EXAMPLE / FLOW

```
Entra app registrations

Angular SPA

Fundraising API

Payment API

Reporting API

Scopes

Roles
```

Test local Angular → local API authentication.

## Azure Stage 2 — Azure SQL

Create:

EXAMPLE / FLOW

```
Azure SQL logical server

FundraisingDb

PaymentDb

ReportingDb
```

Deploy schema.

Then configure:

EXAMPLE / FLOW

```
Entra admin

managed identities later

database users/roles
```

## Azure Stage 3 — Key Vault

Create:

EXAMPLE / FLOW

```
fp-kv-dev
```

Store only genuine secrets.

Use Azure RBAC.

## Azure Stage 4 — Docker

Containerize:

EXAMPLE / FLOW

```
Fundraising API

Payment API

Reporting API

workers
```

Run all containers locally.

## Azure Stage 5 — ACR

Create registry.

Push:

EXAMPLE / FLOW

```
fundraising-api

payments-api

reporting-api
```

## Azure Stage 6 — Container Apps

Create:

EXAMPLE / FLOW

```
Container Apps Environment
```

Deploy APIs.

Initially allow simpler ingress during development.

Verify each service independently.

## Azure Stage 7 — Managed Identity

Enable identities.

Replace:

EXAMPLE / FLOW

```
SQL passwords

Service Bus secrets

Key Vault credentials
```

with passwordless identity where supported.

## Azure Stage 8 — Service Bus

Create:

EXAMPLE / FLOW

```
namespace

topics

subscriptions

DLQ monitoring
```

Implement:

EXAMPLE / FLOW

```
GiftCreated
→ Payment
```

workflow.

## Azure Stage 9 — Background Processing

Deploy:

EXAMPLE / FLOW

```
Outbox worker

Payment consumer
```

Test:

EXAMPLE / FLOW

```
Gift
→ Outbox
→ Service Bus
→ Payment
```

end to end.

## Azure Stage 10 — Reporting Integration

Populate:

EXAMPLE / FLOW

```
ReportingDb
```

using:

EXAMPLE / FLOW

```
events
ETL
or CDC-related process
```

Deploy Reporting API.

## Azure Stage 11 — API Management

Import API OpenAPI definitions.

Expose:

EXAMPLE / FLOW

```
/api/fundraising

/api/payments

/api/reports
```

Add:

EXAMPLE / FLOW

```
token validation

rate limits

routing
```

## Azure Stage 12 — Angular

Deploy Angular.

Configure:

EXAMPLE / FLOW

```
MSAL

Entra client ID

tenant

API scopes

APIM base URL
```

## Azure Stage 13 — Front Door

Add:

EXAMPLE / FLOW

```
custom domain

TLS

WAF

global routing
```

## Azure Stage 14 — GitHub CI/CD

Automate:

EXAMPLE / FLOW

```
build

tests

container build

ACR push

Container Apps deploy

Angular deploy
```

Use:

EXAMPLE / FLOW

```
OIDC
```

for GitHub → Azure identity.

## Azure Stage 15 — Infrastructure as Code

Rebuild dev environment through:

EXAMPLE / FLOW

```
Bicep
```

instead of portal clicks.

Then replicate:

EXAMPLE / FLOW

```
Staging

Production
```

through parameters.

## Azure Stage 16 — Observability

Add:

EXAMPLE / FLOW

```
OpenTelemetry

Application Insights

Log Analytics

dashboards

alerts
```

## Azure Stage 17 — Private Networking

After everything works:

EXAMPLE / FLOW

```
VNet

private endpoints

private DNS

restrict SQL public access

restrict Key Vault public access

private application paths
```

Do not learn networking and application bugs simultaneously if you can avoid it.

## Azure Stage 18 — AKS

Take the exact same Docker images and deploy:

EXAMPLE / FLOW

```
Fundraising API

Payment API

Reporting API
```

to AKS.

Learn:

EXAMPLE / FLOW

Namespaces

Pods

Deployments

Services

Ingress

ConfigMaps

Secrets

Workload Identity

HPA

KEDA

rolling updates

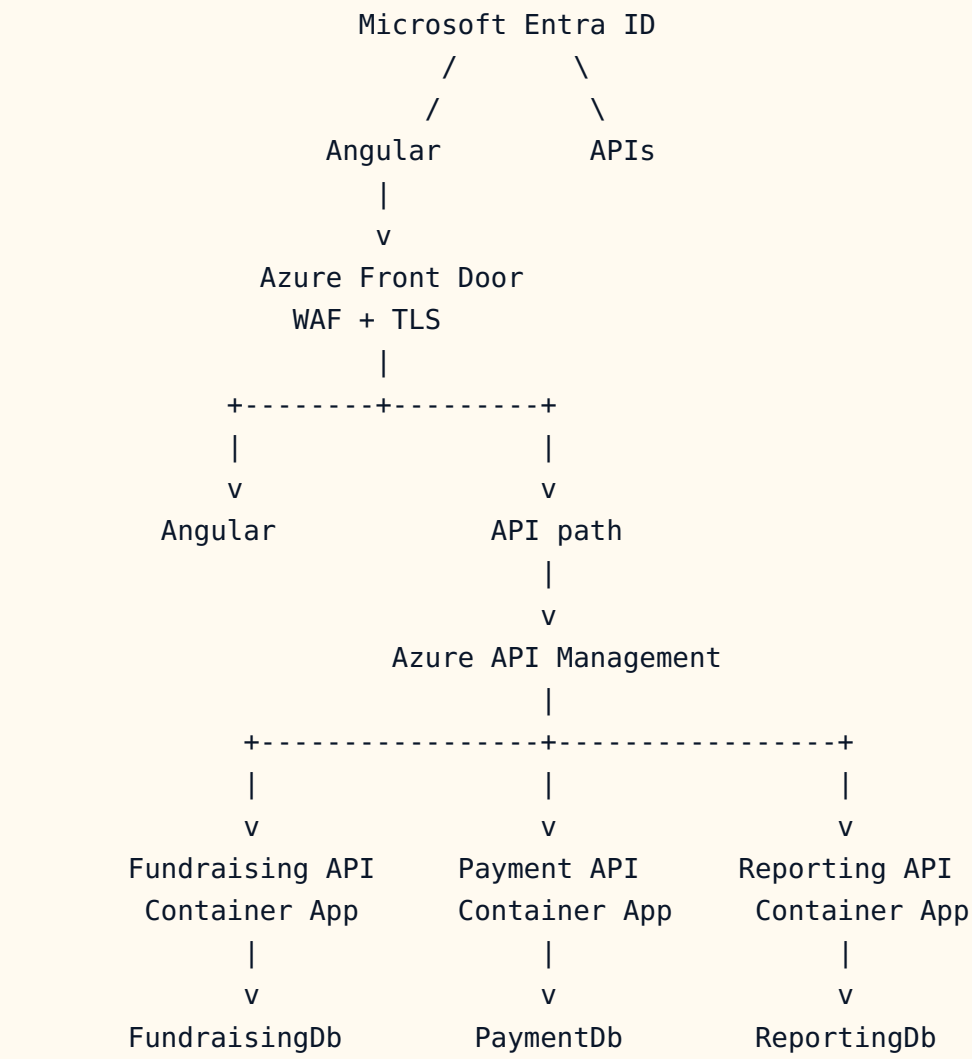
health probes

Compare AKS with Container Apps.

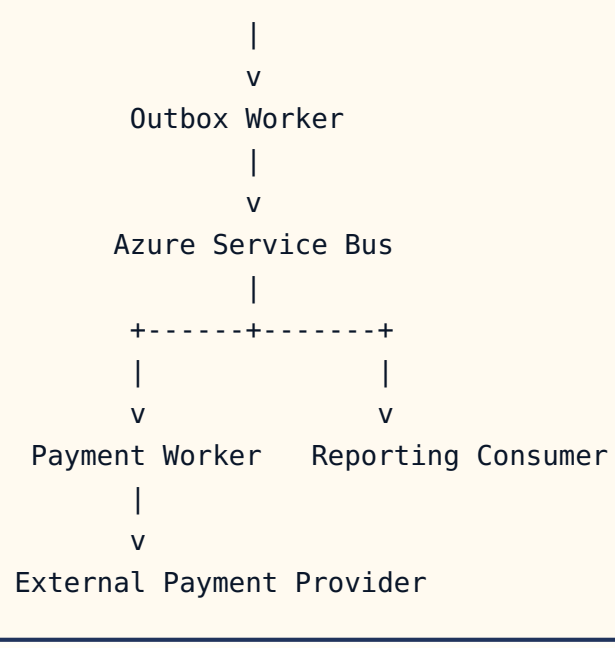
## RECOMMENDED PRODUCTION-LIKE ARCHITECTURE

For this particular project, a strong final learning architecture is:

EXAMPLE / FLOW




EXAMPLE / FLOW — continued




Security:

EXAMPLE / FLOW

```
Managed Identity
    |
    +---- Azure SQL
    |
    +---- Key Vault
    |
    +---- Service Bus
    |
    +---- ACR
```

Observability:

EXAMPLE / FLOW

```
All Components
      |
      v
OpenTelemetry
      |
      v
Application Insights
      |
      v
Azure Monitor
      |
      v
Alerts / Dashboards
```

CI/CD:

EXAMPLE / FLOW

```
GitHub
   |
   v
GitHub Actions
   |
   | OIDC
   v
Microsoft Entra
   |
   v
Azure
   |
   +---- ACR
   |
   +---- Container Apps
   |
   +---- Infrastructure
```

## CONTAINER APPS VS AKS VS APP SERVICE MEMORY RULE

Use this easy rule:

EXAMPLE / FLOW

```
App Service
→ "I mainly have a web app/API
   and want the simplest managed hosting."
```

EXAMPLE / FLOW

```
Container Apps
→ "I have containerized APIs,
   workers, microservices and events,
   but I don't want to operate Kubernetes."
```

EXAMPLE / FLOW

```
AKS
→ "I specifically need Kubernetes
   control and cluster-level capabilities."
```

For this project:

EXAMPLE / FLOW

```
Main deployment
→ Container Apps

Kubernetes learning deployment
→ AKS
```

## AZURE FUNCTIONS MEMORY RULE

EXAMPLE / FLOW

```
Function
→ small/discrete event-driven execution
```

Examples:

EXAMPLE / FLOW

```
timer

Service Bus event

webhook helper

ETL trigger

cleanup
```

Use the current .NET isolated worker model rather than starting new work on the retiring in-process model.

# COMPLETE AZURE TOPIC CHECKLIST

By the end of this project you should understand and demonstrate:

EXAMPLE / FLOW

- ✓ Microsoft Entra tenant
- ✓ Subscription
- ✓ Resource groups
- ✓ Azure regions
- ✓ Dev / Staging / Prod
- ✓ Naming conventions
- ✓ Tags
- ✓ Azure RBAC
- ✓ Entra app registrations
- ✓ SPA app registration
- ✓ Web API app registrations
- ✓ OAuth scopes

EXAMPLE / FLOW — continued

- ✓ App roles
- ✓ Access tokens
- ✓ Managed Identity
- ✓ System-assigned identity
- ✓ User-assigned identity
- ✓ Azure Key Vault
- ✓ Key Vault RBAC
- ✓ Secrets
- ✓ Certificates concepts
- ✓ appsettings
- ✓ Environment variables
- ✓ Azure SQL

EXAMPLE / FLOW — continued

- ✓ Entra SQL authentication
- ✓ Passwordless database connection
- ✓ Private Endpoint
- ✓ VNet
- ✓ Private DNS
- ✓ Docker
- ✓ Dockerfile
- ✓ multi-stage Docker build
- ✓ Docker Compose
- ✓ image tagging
- ✓ Azure Container Registry
- ✓ AcrPull

EXAMPLE / FLOW — continued

- ✓ container image promotion
- ✓ Azure App Service
- ✓ deployment slots
- ✓ Azure Container Apps
- ✓ Container Apps Environment
- ✓ external ingress
- ✓ internal ingress
- ✓ revisions
- ✓ blue/green
- ✓ canary
- ✓ traffic splitting
- ✓ autoscaling

EXAMPLE / FLOW — continued

- ✓ KEDA
- ✓ Container Apps Jobs
- ✓ Azure Functions
- ✓ .NET isolated Functions
- ✓ Flex Consumption
- ✓ Azure Service Bus
- ✓ Queue
- ✓ Topic
- ✓ Subscription
- ✓ Dead-letter queue
- ✓ message retry
- ✓ Outbox

EXAMPLE / FLOW — continued

- ✓ Inbox / idempotent consumer
- ✓ API Management
- ✓ OpenAPI import
- ✓ Entra token validation
- ✓ APIM policies
- ✓ Azure Front Door
- ✓ WAF
- ✓ TLS
- ✓ load balancing concepts
- ✓ Angular deployment
- ✓ Git
- ✓ GitHub

EXAMPLE / FLOW — continued

- ✓ branch / PR workflow
- ✓ GitHub Actions
- ✓ CI
- ✓ CD
- ✓ GitHub environments
- ✓ OIDC / federated deployment
- ✓ Azure RBAC deployment identity
- ✓ Bicep
- ✓ Terraform concepts
- ✓ Azure Developer CLI
- ✓ Infrastructure as Code
- ✓ AKS

EXAMPLE / FLOW — continued

✓ AKS Automatic

✓ AKS Standard

✓ Kubernetes Pod

✓ Deployment

✓ Service

✓ Ingress

✓ Namespace

✓ ConfigMap

✓ Secret

✓ Workload Identity

✓ health probes

✓ Horizontal Pod Autoscaler

EXAMPLE / FLOW — continued

✓ KEDA in Kubernetes

✓ rolling deployment

✓ Application Insights

✓ Azure Monitor

✓ Log Analytics

✓ OpenTelemetry

✓ logs

✓ metrics

✓ traces

✓ distributed tracing

✓ correlation IDs

✓ health checks

EXAMPLE / FLOW — continued

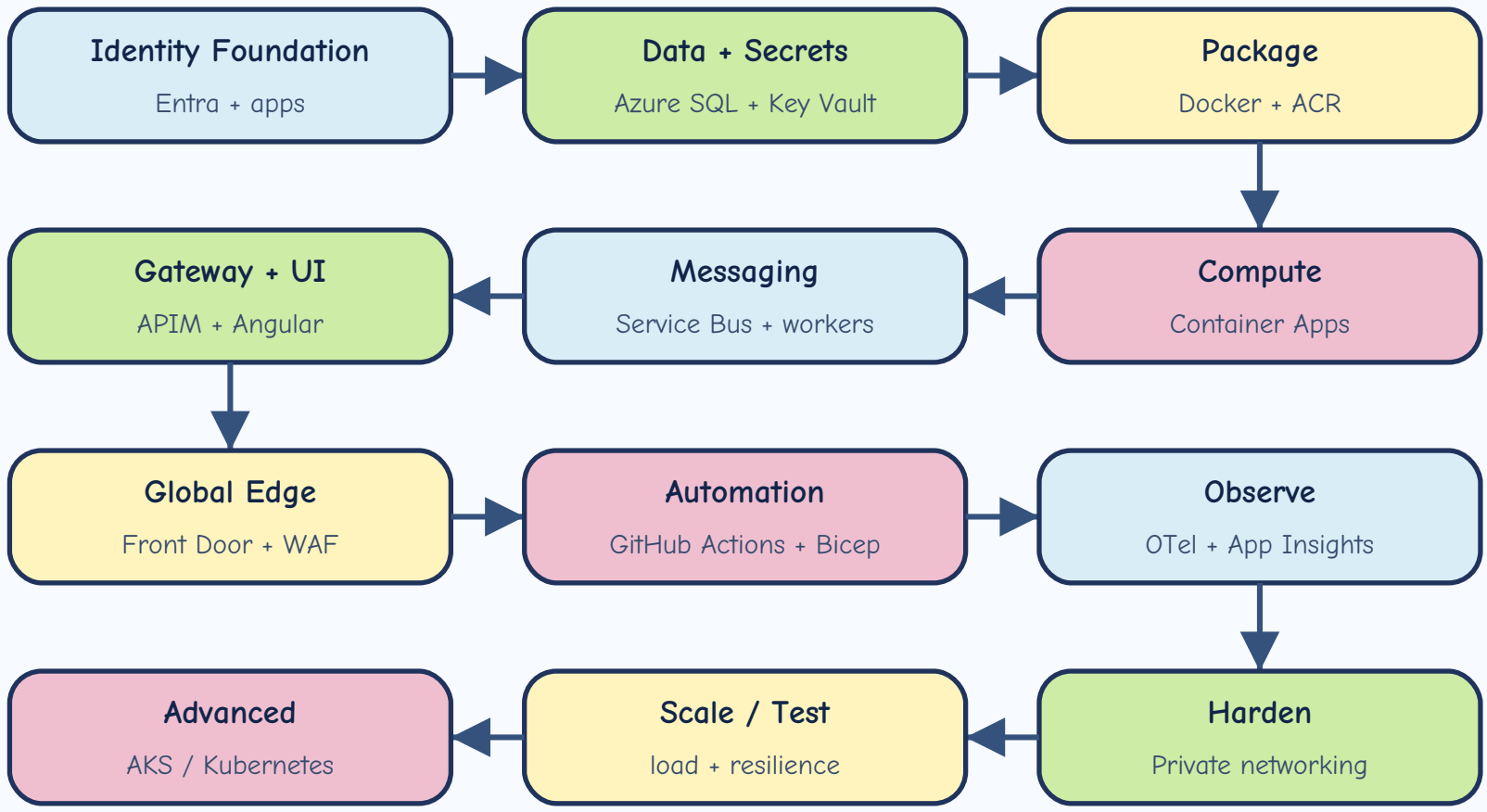
- ✓ alerts
- ✓ dashboards
- ✓ cost management
- ✓ budgets
- ✓ high availability
- ✓ disaster recovery concepts
- ✓ regional deployment concepts
- ✓ secure deployment
- ✓ least privilege
- ✓ production rollback

## FINAL BUILD SEQUENCE

Remember the complete Azure learning sequence:

DIAGRAM

## Recommended Azure Learning & Build Roadmap




Build a working secure PaaS architecture first, automate and observe it, harden networking, then move the same containers to AKS for advanced orchestration.

EXAMPLE / FLOW

1. Entra Tenant / Identity
↓
2. App Registrations
↓
3. Azure SQL
↓
4. Key Vault
↓
5. Docker
↓
6. Azure Container Registry
↓
7. Container Apps
↓
8. Managed Identity
↓
9. Service Bus
↓
10. Workers / Functions
↓
11. Reporting Integration
↓
12. API Management
↓
13. Angular Deployment

EXAMPLE / FLOW — continued

↓
14. Front Door / WAF
↓
15. GitHub Actions CI/CD
↓
16. Bicep Infrastructure as Code
↓
17. Monitoring / OpenTelemetry
↓
18. Private Networking
↓
19. Load / Scale Testing
↓
20. AKS / Kubernetes

## Final Memory Map

EXAMPLE / FLOW

```
IDENTITY
Entra ID
   |
   v
App Registration / Managed Identity
   |
   v
AUTHORIZATION
RBAC / scopes / roles
   |
   v
SECRETS
Key Vault
   |
   v
DATA
Azure SQL
   |
   v
CONTAINERS
Docker
   |
   v
REGISTRY
ACR
```

EXAMPLE / FLOW — continued

```
   |
   v
COMPUTE
Container Apps
   |
   +---- APIs
   +---- Workers
   |
   v
MESSAGING
Service Bus
   |
   v
EVENT-DRIVEN COMPUTE
Functions / Workers
   |
   v
API GATEWAY
API Management
   |
   v
GLOBAL EDGE
Front Door
   |
   v
```

EXAMPLE / FLOW — continued

```
FRONTEND
Angular
   |
   v
OBSERVABILITY
Application Insights
Azure Monitor
OpenTelemetry
   |
   v
AUTOMATION
GitHub Actions
   |
   v
INFRASTRUCTURE AS CODE
Bicep
   |
   v
ADVANCED ORCHESTRATION
AKS / Kubernetes
```

The most important architectural rule is:

! **Use Microsoft Entra ID for identities, managed identities for Azure-to-Azure authentication, Key Vault for genuine secrets, Azure SQL with least-privilege passwordless identities, Docker for packaging, ACR for image storage, Container Apps as the main microservice hosting platform, Service Bus with Outbox/Inbox for service communication, API Management as the API gateway, Front Door for global edge/WAF, GitHub Actions with OIDC for CI/CD, Bicep for repeatable infrastructure, OpenTelemetry/Application Insights for observability, and AKS as the advanced Kubernetes deployment stage when you actually need or want Kubernetes-level control.**

For this project specifically:

EXAMPLE / FLOW

```
Fundraising Service
→ Container App
→ FundraisingDb
→ Outbox
→ Service Bus

Payment Service
→ Container App
→ PaymentDb
→ external provider
→ Outbox
→ Service Bus

Reporting Service
→ Container App
→ ReportingDb

Angular
→ Entra/MSAL
→ Front Door
→ API Management
→ APIs
```

EXAMPLE / FLOW — continued

```
GitHub
→ Actions
→ OIDC
→ ACR
→ Container Apps

All Azure workloads
→ Managed Identity
→ Key Vault / SQL / Service Bus
```

That gives you one continuous path from **local .NET development → Docker → Azure microservices → CI/CD → private cloud security → Kubernetes → production operations** instead of learning Azure services as unrelated topics.